

PrefixQuant: Eliminating Outliers by Prefixed Tokens for Large Language Models Quantization

Mengzhao Chen, Yi Liu, Jiahao Wang, Yi Bin, Wenqi Shao*, Ping Luo*

Abstract—Existing weight-activation quantization methods for Large Language Models (LLMs) primarily address channel-wise outliers but often neglect token-wise outliers, which limits the accuracy of quantized models. In this work, we propose PrefixQuant, a novel quantization method that achieves state-of-the-art performance across various precision levels (W4A4KV4 and W4A8KV4) and granularities (dynamic and static quantization) by effectively isolating token-wise outliers. First, PrefixQuant eliminates token-wise outliers by prefixing outlier tokens in the KV cache, a process that is training-free and highly efficient (*e.g.*, 1 minutes for Llama-3-70B). Second, PrefixQuant introduces new trainable parameters for block-wise training to compensate for quantization error. Our experiments show that PrefixQuant significantly outperforms existing dynamic quantization methods, even under coarser static quantization settings. For instance, PrefixQuant achieves an average accuracy improvement of +3.08 and +2.85 points over SpinQuant (dynamic quantization) on five zero-shot reasoning tasks under dynamic and static quantization settings, respectively, on W4A4KV4 Llama-3-8B. Additionally, we demonstrate up to 2.74 $\times$ prefilling speedup and 2.16 $\times$ decoding speedup for LLMs using W4A4 PrefixQuant. Our code is available at <https://github.com/ChenMnZ/PrefixQuant>.

Index Terms—Large language models, Quantization, Token-wise outliers

I. INTRODUCTION

RECENTLY, Large Language Models (LLMs) [1], [2] demonstrate remarkable capabilities across various tasks. However, their large parameters and computational demands pose significant challenges for deployment. This makes quantization [3]–[5] a crucial technology for reducing memory usage and speeding up inference [6].

Despite advancements, outliers in LLMs activations can lead to significant quantization errors and accuracy degeneration. Many current methods address this by focusing on alleviating channel-wise outliers [8] through techniques like channel-wise scaling [5], [9], [10], mixed-precision quantization [8], [11], Hadamard rotation [7], [12], or channel-level assembly [13]. However, activations of LLMs include not only channel-wise outlier but also some massive activation [14] only occur in a few tokens, and can be termed as token-wise outliers. For example, Figure 1 shows that 2 outlier tokens among 2048 tokens contribute 94.7% of the quantization error. Figure 2(a) provides a more detailed analysis, revealing that a few tokens

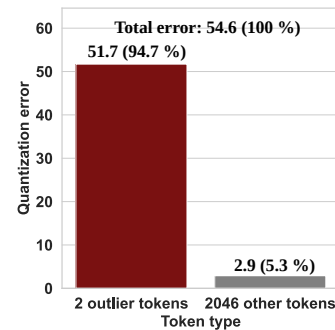


Fig. 1: 4-bit per-token dynamic quantization error in 2048 input context length. Two outlier tokens account for 94.7% of quantization error, while the remaining 2046 tokens contribute only 5.4%. Quantization error is measured in the output of Llama-2-7B 2-nd transformer block through mean square error (MSE).

have extreme values exceeding 1,000, resulting in a quantization error of 54.63. The current state-of-the-art method, Hadamard rotation [7], redistributes outlier values across all channels, reducing the maximum value of outlier tokens from over 1,000 to approximately 15 (see Figure 2 (b)). However, the magnitude of outlier tokens remains hundreds of times larger than normal tokens, leading to a quantization error of 7.88.

In this paper, we propose PrefixQuant, an efficient method to isolate token-wise outliers for more accurate quantization. PrefixQuant is based on a key observation: **Prefixed high-frequency outlier tokens at the beginning of the input sequence constrains token-wise outliers to only occur in the prefixed tokens**. Since prefixed tokens remain consistent across all inputs, PrefixQuant performs offline prefilling of these tokens and stores their KV cache. This stored KV cache can then be reused for all inputs, effectively avoiding token-wise outliers during the forward pass. Furthermore, the detection of prefixed tokens is efficient and does not require any retraining, unlike prior methods [14], [15]. For example, this process completes in just 12 seconds for Llama-2-7B. As shown in Figure 2(c), PrefixQuant effectively eliminates outliers and reduces the quantization error from 7.88 (using QuaRot) to 0.04. Additionally, we introduce a block-wise fine-tuning [5], [16] to compensate for quantization error by jointly training weights and quantization parameters. For static activation quantization, the quantization parameters are

Mengzhao Chen, Jiahao Wang and Peng Xu are with the Department of Computer Science at the University of Hong Kong, China.

Wenqi Shao is with Shanghai AI Laboratory, Shanghai, China.

Yi Liu is with Intel Corporation, Shanghai, China.

Yi Bin is with Tongji University, Shanghai, China.

* Equal Corresponding Authors

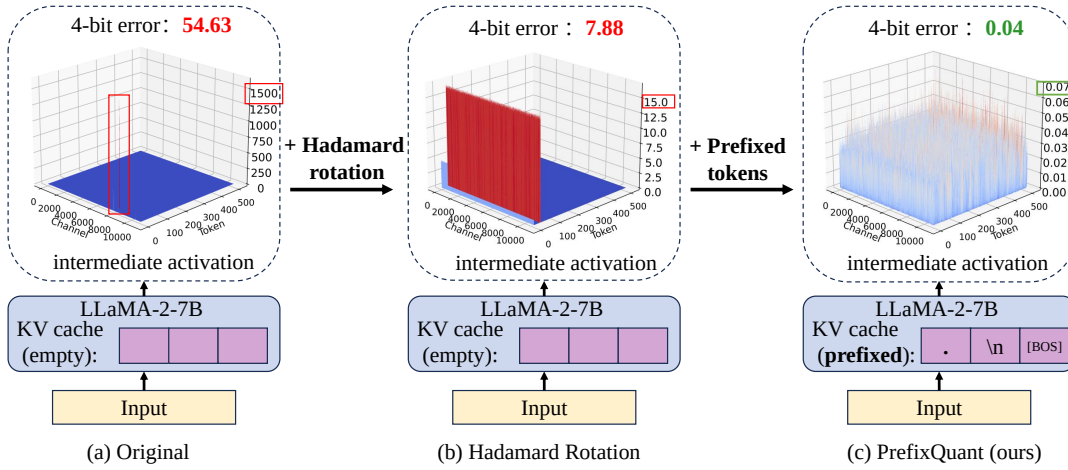


Fig. 2: **Comparison of proposed PrefixQuant with existing methods.** This figure shows the intermediate input activation of the 2-nd down_proj linear layer in Llama-2-7B using different methods. Quantization error is measured in the output of Llama-2-7B 2-nd transformer block through mean square error with 4-bit per-token dynamic quantization. The original distribution has significant outliers larger than 1,500 (left), leading 54.63 quantization error. The previous method with Hadamard rotation [7] reduces outliers to nearly 15 (middle) but still suffers from 7.88 quantization error. We propose PrefixQuant (right), which prefixes some specific tokens in KV cache to isolate outliers, reducing the maximum to nearly 0.07, significantly improving quantization error to 0.04.

TABLE I: Quantization setting of the baselines and PrefixQuant. All group-wise quantization set group size as 128. For PrefixQuant, O1 is the same as existing methods for fair comparisons, and O2 is more efficient than O1 (*i.e.* lower latency).

Method	Weight	Activation	KV Cache
SmoothQuant	per-channel	per-token dynamic	per-token dynamic
Atom	group-wise	group-wise dynamic	group-wise dynamic
QoQ; QuaRot; SpinQuant	per-channel	per-token dynamic	group-wise dynamic
PrefixQuant-O1	per-channel	per-token dynamic	group-wise dynamic
PrefixQuant-O2	per-channel	per-tensor static	per-head static

inherently trainable. However, dynamic activation quantization lacks trainable quantization parameters. To address this, we propose learnable activation clipping to enable training for dynamic activation quantization.

Since PrefixQuant is compatible with various quantization schemes, we introduce two settings for PrefixQuant (see Table I): O1 for dynamic quantization and O2 for static quantization. Experiments show that PrefixQuant significantly outperforms existing methods [7], [9], [17] under the same dynamic quantization setting. Furthermore, PrefixQuant even surpasses prior dynamic quantization methods while utilizing the more efficient static quantization. For instance, PrefixQuant achieves an average accuracy improvement of +3.08 and +2.85 points over SpinQuant (dynamic quantization) on five zero-shot reasoning tasks under dynamic and static quantization settings, respectively, on W4A4KV4 Llama-3-8B. To the best of our knowledge, PrefixQuant is the first method to surpass prior per-token dynamic quantization methods [7], [9], [17] using the coarser per-tensor static quantization. We also benchmark the end-to-end inference of W4A4 quantization, where PrefixQuant achieves a $2.74\times$ prefilling speedup and a $2.16\times$ decoding speedup compared to FP16 models. We hope PrefixQuant inspires future developments in LLM compression.

II. RELATED WORKS

Channel-Wise Outliers. Activation outliers often recur in the same channels across tokens. [8] addresses this by isolating outlier channels with 16-bit precision, while Atom [11] and QUIK [18] adopt similar mixed-precision strategies. Other methods, like SmoothQuant [9], OmniQuant [5], and Outlier Suppression [19], [20], scale activations to weights on a channel-wise basis. QLLM [13] splits outlier channels into sub-channels, and QuaRot [7] redistributes outliers using random Hadamard rotation, later improved by SpinQuant [12], which trains the orthogonal matrix. In contrast, our work focuses on token-wise outliers and is orthogonal to these channel-wise methods.

Token-Wise Outliers. The SoftMax function in self-attention avoids zero attention scores, which creates unnecessary scores for special tokens and results in token-wise outliers [14], [21], [22]. StreamingLLM [21] and LM-infinite [23] keep initial tokens for long-context generation, while our PrefixQuant isolates outliers by carefully selecting prefixed tokens in the KV-cache for quantization. Massive Activation [14] and IntactKV [24] also observe outlier tokens among initial tokens. Massive Activation [14] trains with learnable attention bias, while PrefixQuant isolates outliers without retraining. IntactKV [24] caches either the [BOS] token or the entire

system prompt; PrefixQuant shows that [BOS]-only caching is insufficient and full-prompt caching is redundant. QFeP [25] and CushionCache [26] also explore isolating outlier tokens using prefixed tokens in the KV-cache. However, they depend on time-consuming grid searches to identify these tokens. In contrast, our work shows that outlier token patterns (number and content) remain consistent across different inputs and are easily identified using a small calibration dataset. As a result, PrefixQuant determines prefixed tokens extremely quickly—12 seconds for Llama-3-8B—while CushionCache takes 12 hours, making it 3600x slower.

KV Cache Optimization. As LLM context lengths scale, KV cache optimization [27] becomes a critical research direction for improving inference efficiency. For example, KV cache compression reduces memory use through quantization [28], [29], token merging [30], and low-rank decomposition [31]. Token eviction methods [32], [33] also remove less important tokens to save memory. Other methods [34], [35] redesign KV cache layouts to support flexible allocation and reuse. While existing work mainly focuses on reducing memory use or speeding up inference, PrefixQuant applies prefix operations to the KV cache from a novel, quantization-centric perspective.

III. PRELIMINARIES

Quantization in LLMs involves weight, activation, and KV cache quantization. Weight quantization [16] and KV cache quantization [28] reduce memory usage and speed up memory-bound computations [6]. Combining weight and activation quantization enables low-bit matrix manipulation to accelerate computation-bound tasks [6]. Specifically, the quantization process is:

$$\mathbf{X}_{\text{INT}} = \text{clamp} \left(\left\lfloor \frac{\mathbf{X}}{s} \right\rfloor + z, 0, 2^N - 1 \right), \quad (1)$$

$$\text{where } s = \frac{\gamma \max(\mathbf{X}) - \beta \min(\mathbf{X})}{2^N - 1}, z = -\left\lfloor \frac{\beta \min(\mathbf{X})}{s} \right\rfloor \quad (2)$$

where $\lfloor \cdot \rfloor$ denotes rounding operation, N is the target bit number, $\mathbf{X}_{\text{INT}}$ and $\mathbf{X}$ are the quantized integer and full-precision tensor, respectively. s and z are quantization parameters, for the scale factor and zero point, respectively. $\gamma \in [0, 1]$ and $\beta \in [0, 1]$ are clipping factors.

Quantization Granularities. Granularity determines how scale factors (and zero points) are shared across a tensor. Finer granularity tends to improve accuracy at the cost of more compute and memory. Typical options are: (i) Per-tensor: a single scale for the whole tensor. (ii) Per-channel/token: a scale broadcast along the channel or token axis. (iii) Group-wise (or block-wise): split one axis into contiguous groups of size k , assigning one scale to each group.

Dynamic and Static. Activation quantization is divided into dynamic and static quantization based on how quantization parameters are calculated. Specifically, dynamic quantization calculates s and z online during inference, offering better adaptability to different distributions. In contrast, static quantization precomputes s and z offline through calibration datasets, leading to more efficient inference and more feasible operator fusion [36].

Initialization of Quantization Parameters. The classical approach uses max-min initialization, where both γ and β are set to 1. To better balance clipping error and rounding error [4], [17], we initialize γ and β using MSE-based grid search for both weight and activation quantization in our experiments. Specifically, for per-token dynamic quantization, γ and β are shared across all tokens within the same layer. For per-tensor static quantization, we directly perform a grid search for the quantization parameters s and z instead of optimizing the clipping factors.

Block-wise Fine-tuning Recent studies show that block-wise fine-tuning [5], [16] improves performance by accounting for inter-layer interactions [37]. To further enhance the performance of quantized models, we fine-tune each transformer block sequentially using a mean squared error (MSE) loss. Specifically, we introduce trainable parameters for the activation quantizer to balance the rounding and clipping errors in quantization. For dynamic activation quantization in PrefixQuant-O1, we set the tensor-wise clipping factors (γ and β) as trainable. For static activation quantization in PrefixQuant-O2, the quantization parameters (s and z) are inherently trainable. For weight quantization, we follow the approach of EfficientQAT [16], enabling the training of all weights and weight quantization parameters.

Hadamard Rotation. Random Hadamard rotation [7], [12] addresses channel-wise outliers. Our method focus on removing token-wise outliers. Therefore, We build our method upon the Hadamard rotation technique [7]. The Hadamard matrix $\mathbf{H}$ is an orthogonal matrix with $\mathbf{H}\mathbf{H}^T = \mathbf{I}$, and its entries are $\{+1, -1\}$ at the same scale. Following SpinQuant [12], we use four rotations R_1 – R_4 (Figure 3). Rotations on the input activations of q/k/v/gate/up_proj (R_1) and on o_proj (R_2) are absorbable: they merge into the preceding linear layer and add no inference cost. Other rotations run online: the output activations of q_proj and k_proj after RoPE [38] (R_3), and the input activations of down_proj (R_4). We implement these online rotations with the Walsh-Hadamard transform, which introduces negligible overhead. Unless stated otherwise, we enable all rotations (R_1 – R_4) for weight-activation quantization, and only absorbable rotations (R_1 , R_2) for weight-only quantization.

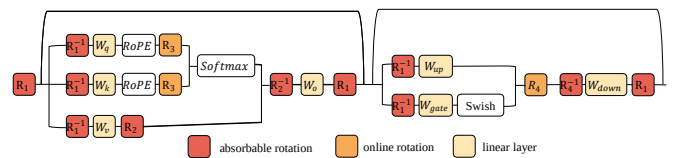


Fig. 3: Illustrate of hadamard rotation within a transformer block of Llama [1] model.

IV. PREFIXQUANT

In this section, we present the proposed PrefixQuant methods. Sec.IV-A describes the characteristics of outlier tokens in LLMs. Sec.IV-B proposes a solution to isolate these outlier tokens, creating a more quantization-friendly distribution.

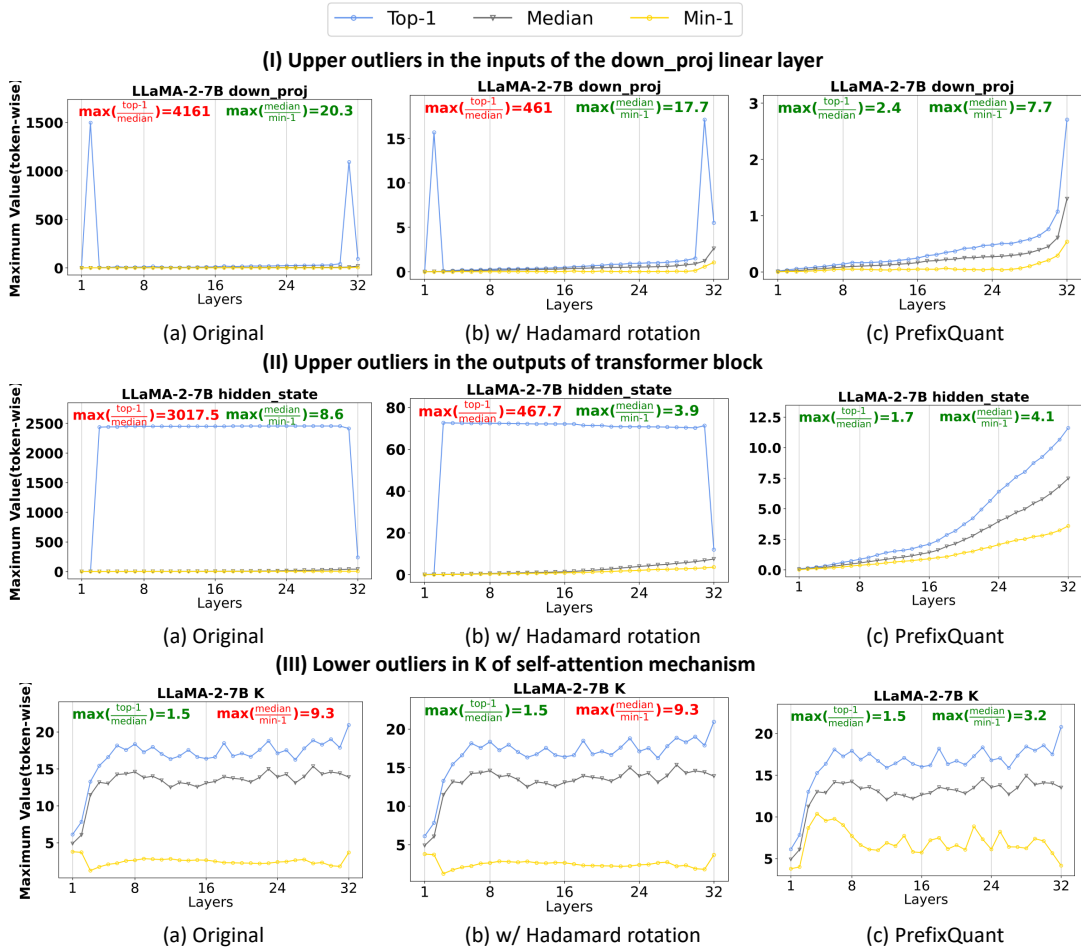


Fig. 4: Example of token-wise outliers. We present (I)(II) upper outliers and (III) lower outliers. Top-1, Medium, Min-1 indicate the largest, median, and smallest values among token-wise maximum values, respectively. We also calculate the the ratios of $\frac{\text{Top-1}}{\text{Median}}$ and $\frac{\text{Median}}{\text{Min-1}}$ in each layer, and report the maximum ratio across all layers . A lower ratio indicates a more uniform distribution. we take Llama-2-7B as an example here, more visualizationsn about other models can be find in Appendix.

A. Deep Exploration of Outlier Tokens

Both channel-wise and token-wise outliers can cause significant quantization error. While channel-wise outliers have been thoroughly explored and addressed in prior research [7], this discussion focuses on token-wise outliers.

Definition of Outlier Token. Let $\mathbf{X} \in \mathbb{R}^{T \times C}$ represent the absolute values of a token sequence, where T is the number of tokens and C is the dimension size. We compute the token-wise maximum values $\mathbf{M} \in \mathbb{R}^T$, where each element M_i indicates the maximum value of the i -th token. The outlier degree of a token is then measured by comparing its maximum value to the median of $\mathbf{M}$:

$$R_i = \frac{M_i}{\text{median}(\mathbf{M})}. \quad (3)$$

Then, the i -th token is classified as an outlier if R_i deviates significantly from 1 (i.e., either much larger or much smaller than 1). Specifically, we define an *upper outlier token* when $R_i > \eta_1$ and a *lower outlier token* when $R_i^{-1} > \eta_2$. In our experiments, we set $\eta_1 = 64$ and $\eta_2 = 8$.

Visualization of Outlier Tokens. Let top-1, median, and min-1 denote the maximum, median, and minimum of $\mathbf{M}$, respectively. To better illustrate the outlier degree, we further define $\max\left(\frac{\text{top-1}}{\text{median}}\right)$ and $\max\left(\frac{\text{median}}{\text{min-1}}\right)$ as the maximum R_i and maximum R_i^{-1} across different layers, respectively. Following the definition of R_i in Eq. (3), a larger $\max\left(\frac{\text{top-1}}{\text{median}}\right)$ indicates the presence of extreme upper outliers, while a larger $\max\left(\frac{\text{median}}{\text{min-1}}\right)$ reflects the presence of extreme lower outliers. Specifically, we identify the following outlier tokens:

1) Upper outlier tokens in inputs of down_proj layers and outputs of transformer blocks. As shown in Figure 4(I.a), the input activations of the down_proj layers exhibit significant upper outliers, with $\max\left(\frac{\text{top-1}}{\text{median}}\right) = 4161$. Although Hadamard rotation (Figure 4(I.b)) reduces this ratio to 461, it still indicates a large gap compared to normal tokens. A similar phenomenon is observed in the outputs of transformer blocks, as shown in Figure 4(II). These outliers not only lead to larger quantization errors but also cause instability during block-wise fine-tuning.

2) Lower outlier tokens in Q/K. In Figure 4(II.a), K

displays a distinct outlier pattern compared to the inputs of linear layers. Instead of having large magnitudes, some tokens exhibit extremely small values. Specifically, $\mathbf{K}$ has $\max\left(\frac{\text{top-1}}{\text{median}}\right) \approx 1.5$, but $\max\left(\frac{\text{median}}{\text{min-1}}\right) > 9$. Furthermore, as shown in Figure 4(II.b), Hadamard rotation does not mitigate these lower outliers.

Additionally, we observe that both upper outlier tokens and lower outlier tokens correspond to tokens at the same position in the sequence, but they exhibit different patterns in different modules. Therefore, we focus on analyzing upper outlier tokens due to their stronger prominence and ease of detection.

Characters of outlier tokens. We further investigate the characteristics of these outlier tokens, including the number of outlier tokens in an input sequence, their positions, and their content (text):

- **Number:** We determine the number of outlier tokens in a small calibration dataset. Specifically, we compute the average outlier token count $\mathbf{O} \in \mathbb{R}^b$ for each transformer block according to compare Eq (3) with the outlier threshold η_1 , where b is the total number of transformer blocks. Since outlier tokens are nearly consistent across layers that contain them, we simply set the number of outlier tokens as $o = \lceil \max(\mathbf{O}) \rceil$. Consistent with Massive Attention [14], we find that outlier tokens appear in only a small fraction of positions (e.g. 2 for Llama-2-7B) within the input sequence, as shown in Figure 5a.
- **Position:** We observe that the initial tokens are outlier tokens across almost all models, aligning with findings on attention sinks [21]. Additionally, Figure 5c shows that, apart from the initial tokens, some other tokens near the beginning of the sequence are also outlier tokens. Unlike outlier channels, which occur at fixed channel indices [8], the positions of outlier tokens depend on the input sequence and vary significantly. As a result, it is not feasible to identify outlier tokens offline for mixed-precision quantization.
- **Content (text):** Initial tokens are consistently outlier tokens, regardless of their content. Thus, we focus on outlier tokens that are not initial tokens to analyze their content. Some models, such as Llama-3-8B and Qwen-2-7B, exhibit outlier tokens only at the initial positions. However, certain models display outlier tokens not only at the start of the input sequence but also in low-semantic tokens. For example, Llama-2-7B shows outlier tokens in both initial and delimiter tokens (e.g., "." or "\n"), as illustrated in Figure 5b. Notably, tokens corresponding to the same text may exhibit different patterns depending on their position in the sequence. For instance, low-semantic tokens may behave as outlier tokens at the front of the sequence but appear as normal tokens in other positions.

B. Prefixed Outliers

Given that the number of outlier tokens is limited and they typically occur at the beginning of the input sequence, We propose a method to prefix high-frequency outlier tokens in the input sequence. This approach constrains outlier tokens

TABLE II: Prefixed tokens in KV cache across different models. [BOS] indicates the special token for beginning of sequence(e.g. "<s>" for Llama-2 and "[begin_of_text]" for Llama-3). Note that the following "-" represents space.

Model	Prefixed token	
	Number	Content
Llama-2-7B	3	.\n[BOS]
Llama-2-13B	3	the.[BOS]
Llama-2-70B	4	\n"[BOS]
Llama-3-8B(-Instruct)	1	[BOS]
Llama-3-70B(-Instruct)	3	..[BOS]
Mistral-v0.3-7B	4	\n.to[BOS]
Qwen-2-7B	1	[BOS]

to the prefixed tokens. Furthermore, these prefixed tokens can be directly stored in the KV cache, enabling more efficient computation, as shown in Figure 2.

What token is added as a prefix. Given that these outlier tokens typically appear only among initial or low-semantic tokens, we opt to designate the top- o most frequent outlier tokens as prefixed tokens, avoiding the time-consuming grid-search method [26] across the entire vocabulary. Specifically, o denotes the number of outlier tokens defined in Sec. IV-A, and the outlier-token frequency is measured by examining the outputs of each Transformer block. Note that initial tokens are excluded from this calculation, as they are outliers regardless of their content. For specific models like Llama-3-8B and Qwen-2-7B, where outlier tokens occur solely as initial tokens, we assign the prefix token "[BOS]". To ensure uniformity, we append "[BOS]" as the final prefixed token for all models. The prefixed tokens for each model appear in Table II.

Computation of prefixed tokens in KV cache. In the autoregressive inference pipeline of LLMs, we directly store these prefixed tokens in the KV cache to prevent new outlier tokens from being generated during inference. Specifically, given the input query, key, and value matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{T \times C}$, the self-attention mechanism with prefixed tokens in the KV cache is formulated as:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}; \mathbf{k}', \mathbf{v}') = \text{Softmax}\left(\frac{\mathbf{Q} [\mathbf{K}^T \mathbf{k}']}{\sqrt{d}}\right) \begin{bmatrix} \mathbf{V} \\ \mathbf{v}'^T \end{bmatrix} \quad (4)$$

Here, $\mathbf{k}', \mathbf{v}' \in \mathbb{R}^{o \times C}$ represent the prefixed tokens stored in the KV cache. We calculate $\mathbf{k}'$ and $\mathbf{v}'$ once during a prefilling step using the full-precision model. These tokens are then saved in the KV cache and reused by quantized models during inference. Setting prefixed outlier tokens in the KV cache prevents upper outliers in the inputs to the down_proj layers, though lower outliers persist in these tokens. Thus, we keep the prefixed tokens in full precision within the KV cache to maintain performance with static quantization.

Distribution changing after setting prefixed tokens.

As shown in Figure 4, prefixing outlier tokens in the KV cache significantly improves the distribution. Specifically, the $\max\left(\frac{\text{top-1}}{\text{median}}\right)$ ratio of the down_proj inputs decreases from 461 to 2.4 and the $\max\left(\frac{\text{median}}{\text{min-1}}\right)$ ratio of $\mathbf{Q}/\mathbf{K}$ decreases from > 9 to < 3.5 .

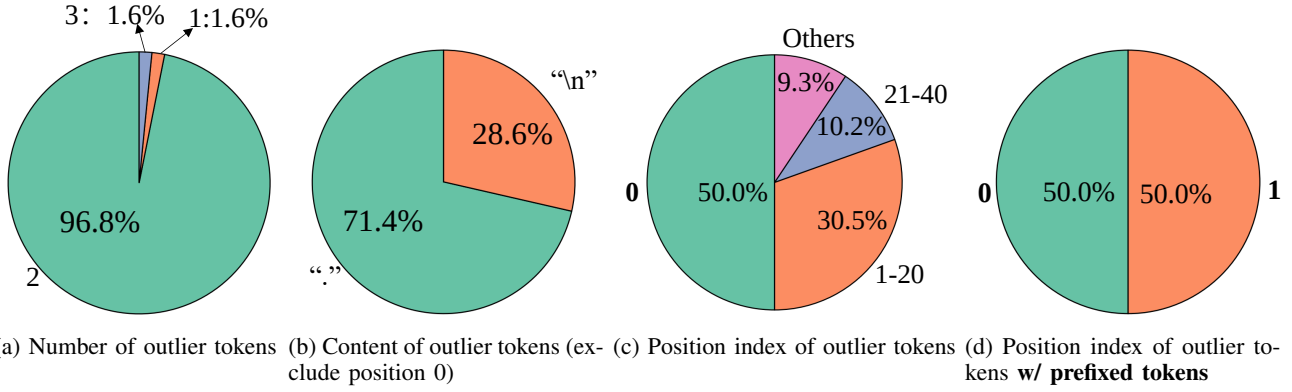


Fig. 5: **Explorations of outlier tokens in Llama-2-7B.** (a) Outlier token only exits in nearly 2 positions in the overall input sequence. (b) Excluding token in position 0, outlier tokens only exits in ‘.’ or ‘\n’ tokens. (c) Outlier tokens consistently occur in the starting token (position 0) and another front but un-predictable position index. (d) Prefixing the input sequence with high-frequency outlier tokens (‘.\n’) can constraint the outlier tokens only exit in position 0 and 1.

V. EXPERIMENTS

Sec. V-A details the experimental setup. Sec. V-B compares the proposed PrefixQuant with existing methods. Sec. V-C evaluates the effectiveness of each proposed component. Additionally, Sec. V-D presents the actual speedup of the proposed method in W4A4 precision.

A. Setups

Baseline. PrefixQuant is a versatile method applicable to any precision. We conduct experiments on two mainstream precisions: W4A8KV4, and W4A4KV4. The detailed quantization settings are illustrated in Table I. PrefixQuant-O1 is consistent with existing methods for fair comparisons, and PrefixQuant-O2 targets to push the limitation of more efficient static quantization. We compare against QuaRot [7], Atom [11], DuQuant [39], QoQ [17], SmoothQuant [9], SpinQuant [12], OSTQuant [40], FlatQuant [41], and EfficientQAT, with data sources as follows:

- **QuaRot:** Evaluated with the Pile calibration dataset. Llama-2 W4A4KV4 results are taken from QoQ [17]; others are reproduced using the official code.
- **DuQuant:** Reproduced with the Pile calibration dataset using the official code.
- **Atom:** Results sourced from QoQ [17].
- **QoQ:** Pile calibration dataset. Llama-2 results are from QoQ [17]; Llama-3 results are reproduced with the official code.
- **SmoothQuant:** Pile calibration dataset; all results reproduced using QoQ’s open-source code [17].
- **SpinQuant:** Reproduced with the official code and pre-trained rotation matrix; trained directly on WikiText2.
- **FlatQuant / OSTQuant:** Results taken from their original papers [40], [41], trained directly on WikiText2.
- **EfficientQAT:** Reproduced with the official code and provided pre-quantized models.

Models and datasets. We evaluate PrefixQuant on the Llama-2, Llama-3, Llama-3-Instruct families, Mistral-7B-v0.3,

TABLE III: **W4A4KV4 results on Llama models.** “PPL” indicates WikiText2 perplexity measured with context length 2048. “Acc.” indicates the average zero-shot accuracy on 5 common-sense reasoning tasks. Grayed results use Wikitext2 as calibration dataset, leading the Wikitext2 perplexity is not very reliable.

Model	Method	PPL	Acc.
2-7B	FP16	5.47	69.04
	Atom	6.12	59.73
	QuaRot	6.19	64.69
	DuQuant	6.20	66.25
	SpinQuant	5.95	65.35
	FlatQuant	5.79	66.98
	OSTQuant	5.91	66.56
	PrefixQuant-O1	5.93	66.74
	PrefixQuant-O2	6.01	66.37
2-13B	FP16	4.88	71.73
	Atom	5.31	63.51
	QuaRot	5.45	69.01
	DuQuant	5.39	69.13
	SpinQuant	5.24	69.24
	FlatQuant	5.12	70.44
	OSTQuant	5.25	69.14
	PrefixQuant-O1	5.24	70.05
	PrefixQuant-O2	5.32	70.36
2-70B	FP16	3.32	76.72
	Atom	3.73	67.52
	QuaRot	3.83	75.43
	DuQuant	3.77	74.75
	SpinQuant	3.70	75.19
	FlatQuant	3.55	76.02
	OSTQuant	3.59	75.70
	PrefixQuant-O1	3.62	76.23
	PrefixQuant-O2	3.81	75.48
3-8B	FP16	6.14	72.71
	Atom	7.76	-
	QuaRot	8.41	65.15
	DuQuant	8.14	67.13
	SpinQuant	7.56	68.23
	FlatQuant	6.98	70.89
	OSTQuant	7.29	69.96
	PrefixQuant-O1	7.26	71.31
	PrefixQuant-O2	7.43	71.08
3-70B	FP16	2.85	80.03
	QuaRot	6.82	68.39
	DuQuant	5.67	74.89
	FlatQuant	3.78	79.02
	OSTQuant	4.01	77.13
	PrefixQuant-O1	4.16	77.08
	PrefixQuant-O2	4.41	77.18

Deepseek-llm-7B, and Qwen-2-7B models. Following previous literature [5], [17], we assess PrefixQuant quantized models on language modeling and zero-shot tasks. Specifically, We

TABLE IV: **W4A8KV4 results on Llama models.** “PPL” indicates WikiText2 perplexity measured with context length 2048. “Acc.” indicates the average zero-shot accuracy on 5 common-sense reasoning tasks.

Model	Method	PPL	Acc.
2-7B	FP16	5.47	69.04
	SmoothQuant	6.07	65.34
	OmniQuant	5.87	66.94
	QoQ	5.75	67.22
	QuaRot	5.73	67.11
	PrefixQuant-O1	5.67	68.04
2-13B	FP16	4.88	71.73
	QoQ	5.12	70.56
	QuaRot	5.07	69.96
	PrefixQuant-O1	5.05	71.25
	PrefixQuant-O2	5.07	71.25
2-70B	FP16	3.32	76.72
	QoQ	3.52	75.91
	QuaRot	3.46	76.31
	PrefixQuant-O1	3.44	76.82
	PrefixQuant-O2	3.50	76.50
3-8B	FP16	6.14	72.71
	QoQ	6.89	71.35
	QuaRot	6.80	71.68
	PrefixQuant-O1	6.59	72.57
	PrefixQuant-O2	6.62	72.46
3-70B	FP16	2.85	80.03
	QoQ	4.36	78.12
	QuaRot	3.73	78.52
	PrefixQuant-O1	3.37	78.50
	PrefixQuant-O2	3.43	78.70

evaluate on WikiText2 [42] with a 2048-token context, where perplexity is computed as $PPL = \exp(\frac{1}{N} \sum_{t=1}^N -\log p(x_t | x_{<t}))$, with N the number of tokens in the evaluation set and $p(x_t | x_{<t})$ the model’s predicted probability of token x_t given its preceding context. We also evaluate on 5 zero-shot reasoning tasks, including PIQA [43], ARC [44], HellaSwag [45], and WinoGrande [46]. We also test models on more challenge zero-shot MMLU [47]. All accuracy are measured through `lm_eval v0.4.2` [48]. For accuracy, we report `acc` for WinoGrande and `acc_norm` for HellaSwag, Arc_Challenge, Arc_Easy, and PIQA, following Qserve [17].

Grid Search Initialization Setting. We initialize the quantization parameters through grid search on 8 Pile [49] samples with a 1024 sequence length. We minimize the layer outputs for fine-grained quantization (per-channel/per-head) and block outputs for per-tensor quantization.

Fine-Tuning Setting. During fine-tuning, we optimize block output mean square error following existing works [5], [16]. The dataset for fine-tuning consists of 512 samples from Pile with a 1024 context length. The learning rates for quantization parameters (step sizes) and full-precision weights are set to $5e-5$ and $5e-6$, respectively, and to $2e-5$ and $2e-6$ for Llama-3-70B(-Instruct) models. The fine-tuning batch size is set to 4, and the number of epochs is set to 10 for W4A8KV4 and 20 for W4A4KV4.

B. Comparison Results

Performance of W4A4KV4 on Llama Models. Table III summarizes the evaluation results for the W4A4KV4 configuration across various Llama models. The proposed PrefixQuant method consistently delivers superior performance compared to existing approaches. For instance, on the Llama-3-8B model under dynamic quantization, PrefixQuant-O1 achieves

TABLE V: **MMLU average accuracy (zero-shot) on Llama-3-8B.**

Method	Precision	MMLU Acc.
-	FP16	62.07
QuaRot	W4A4KV4	34.25
DuQuant	W4A4KV4	50.77
SpinQuant	W4A4KV4	51.93
PrefixQuant-O1	W4A4KV4	56.00
PrefixQuant-O2	W4A4KV4	54.65
QuaRot	W4A8KV4	38.37
DuQuant	W4A8KV4	58.01
SpinQuant	W4A8KV4	58.25
PrefixQuant-O1	W4A8KV4	60.49
PrefixQuant-O2	W4A8KV4	59.20

a notable improvement over DuQuant, reducing WikiText2 perplexity by 0.88 (from 8.14 to 7.26) and increasing average zero-shot accuracy by 4.18 points (from 67.13 to 71.31). Similarly, the more efficient PrefixQuant-O2, designed for static quantization, outperforms DuQuant with a perplexity reduction of 0.71 (from 8.14 to 7.43) and an accuracy gain of 3.95 points (from 67.13 to 71.08). We also compare PrefixQuant with two concurrent works: FlatQuant and OSTQuant. PrefixQuant consistently surpasses OSTQuant across all tested models. The comparison with FlatQuant yields mixed results: PrefixQuant achieves superior performance on Llama-2-70B and Llama-3-8B, while FlatQuant performs better on other models. However, the two methods are compatible, as PrefixQuant specifically targets token-wise outliers. Additionally, PrefixQuant introduces an additional Hadamard rotation only in the down_proj linear layer, whereas FlatQuant requires extra matrix manipulation for every linear layer. These results demonstrate the effectiveness of PrefixQuant in balancing perplexity and accuracy across different model sizes and quantization settings.

Performance of W4A8KV4 on Llama Models. Table IV presents the evaluation results of the W4A8KV4 configuration across various Llama models, comparing our proposed methods, PrefixQuant-O1 and PrefixQuant-O2, against baselines FP16, QoQ [17], and QuaRot. Across all tested models, PrefixQuant-O1 and PrefixQuant-O2 consistently achieve better performance than QoQ and QuaRot, while closely approaching the FP16 baseline. For example, in the Llama-3-8B model, PrefixQuant-O1 reduces perplexity by 0.30 (from 6.89 to 6.59) and improves accuracy by 1.22 points (from 71.35 to 72.57) compared to QoQ. Similarly, PrefixQuant-O2 lowers perplexity by 0.27 (to 6.62) and increases accuracy by 1.11 points (to 72.46) over QoQ. Similar trends are observed across other model sizes, with PrefixQuant-O1 often showing the strongest results, followed closely by PrefixQuant-O2. These improvements highlight the effectiveness of our methods in maintaining model quality under the W4A8KV4 setting.

Performance on Other Models. Table VI shows the results of PrefixQuant-O1 and PrefixQuant-O2 on Mistral-v0.3-7B, Qwen-2-7B, and instruction-tuned Llama-3-8B-Instruct and Llama-3-70B-Instruct under W4A8KV4 and W4A4KV4 settings. Overall, PrefixQuant-O1 and O2 under W4A8KV4 perform close to FP16, while W4A4KV4 impacts quality more, proving their effectiveness across diverse models.

Performance on MMLU. Table V displays the zero-

TABLE VI: **Results of proposed PrefixQuant-O1/O2 on other models.** “PPL” indicates WikiText2 perplexity measured with context length 2048. “Acc.” indicates the average zero-shot accuracy on 5 common-sense reasoning tasks.

Model	Precision	PPL	Acc.
Mistral-v0.3-7B	FP16	5.32	73.43
	W4A8KV4-O1	5.49	73.07
	W4A8KV4-O2	5.51	73.19
	W4A4KV4-O1	5.76	71.48
	W4A4KV4-O2	5.79	71.45
Qwen-2-7B	FP16	7.14	72.72
	W4A8KV4-O1	7.36	71.65
	W4A8KV4-O2	8.04	71.65
	W4A4KV4-O1	7.76	70.38
	W4A4KV4-O2	8.37	69.22
Deepseek-llm-7B	FP16	6.84	68.23
	W4A8KV4-O1	7.10	67.50
	W4A8KV4-O2	7.03	67.91
	W4A4KV4-O1	7.36	66.17
	W4A4KV4-O2	7.44	65.47
Llama-3-8B-Instruct	FP16	8.29	72.55
	W4A8KV4-O1	8.73	71.00
	W4A8KV4-O2	8.74	70.97
	W4A4KV4-O1	9.01	69.75
	W4A4KV4-O2	8.96	69.97
Llama-3-70B-Instruct	FP16	5.33	77.96
	W4A8KV4-O1	5.82	78.49
	W4A8KV4-O2	5.96	78.41
	W4A4KV4-O1	6.49	77.66
	W4A4KV4-O2	6.80	77.29

TABLE VII: **Comparisons in long-context scenario of Llama-3-8B.** We report the WikiText2 perplexity with context length 8192. We do not report SpinQuant results because it overfits to WikiText2 datasets.

Method	Precision	PPL
-	FP16	5.54
QuaRot	W4A8KV4	6.79
DuQuant	W4A8KV4	6.19
PrefixQuant-O1	W4A8KV4	5.94
PrefixQuant-O2	W4A8KV4	6.04
QuaRot	W4A4KV4	8.41
DuQuant	W4A4KV4	7.27
PrefixQuant-O1	W4A4KV4	6.58
PrefixQuant-O2	W4A4KV4	6.82

shot MMLU average accuracy for Llama-3-8B across different methods and precision settings (FP16, W4A8KV4, and W4A4KV4). MMLU, being a more challenging validation set, experiences greater performance drops during quantization compared to the FP16 baseline of 62.07. In W4A8KV4, PrefixQuant-O1 achieves the best accuracy at 60.49, beating SpinQuant (58.25) by 2.24 points and DuQuant (58.01) by 2.48 points. PrefixQuant-O2 follows with 59.20, outperforming SpinQuant by 0.95 points. In W4A4KV4, where degradation is more severe, PrefixQuant-O1 reaches 56.00, surpassing SpinQuant (51.93) by 4.07 points and DuQuant (50.77) by 5.23 points, while PrefixQuant-O2 scores 54.65, exceeding SpinQuant by 2.72 points.

Performance on Long-Context Scenarios. PrefixQuant-O1 uses a shared clipping factor for each layer, while PrefixQuant-O2 further shares a scaling factor across the entire tensor. A longer input context implies that more activations share the same clipping or scaling factor. This raises a concern about whether PrefixQuant remains effective in long-context scenarios. Table VII presents the WikiText2 perplexity (PPL) for Llama-3-8B with a context length of 8192, comparing

TABLE VIII: Comparisons with CushionCache using W8A8 per-tensor static quantization. “PPL” indicates WikiText2 perplexity measured with context length 2048. “Prefix Time” represent the time to find the prefixed tokens. Note that we remove rotation from PrefixQuant in this table for fair comparisons.

Model	Method	PPL	Prefix Time
Llama-2-7B	CushionCache	5.98	2.68 h
Llama-2-7B	PrefixQuant-O2	5.73	11 s
Llama-3-8B	CushionCache	7.41	12.09 h
Llama-3-8B	PrefixQuant-O2	7.22	12 s

different methods under W4A8KV4, and W4A4KV4 precision settings. SpinQuant results are excluded due to overfitting on WikiText2. In W4A8KV4, PrefixQuant-O1 achieves the lowest PPL at 5.94, improving over DuQuant (6.19) by 0.25, while PrefixQuant-O2 scores 6.04, beating DuQuant by 0.15. In W4A4KV4, PrefixQuant-O1 again performs best with a PPL of 6.58, surpassing DuQuant (7.27) by 0.69, followed by PrefixQuant-O2 at 6.82, ahead of DuQuant by 0.45. These results show that PrefixQuant-O1 and PrefixQuant-O2 handle long-context scenarios well, consistently outperforming QuaRot and DuQuant in both precision settings. This suggests strong performance even when sharing clipping or scaling factors across longer inputs.

Comparisons with CushionCache. CushionCache [26] enhances quantization performance by using grid-searched tokens in the KV cache. We compare PrefixQuant with CushionCache in Table VIII. For a fair comparison, we disable rotation in PrefixQuant and apply W8A8 per-tensor static quantization. We keep block-wise fine-tuning, as CushionCache also uses fine-tuning. Table VIII shows that PrefixQuant achieves lower WikiText-2 perplexity than CushionCache. For example, with Llama-3-8B, PrefixQuant records a PPL of 7.22, better than CushionCache’s 7.41, while requiring only 12 seconds compared to 12.09 hours for finding prefixed tokens. These results prove PrefixQuant is more effective and significantly faster than CushionCache.

TABLE IX: Weight-only quantization results. “g” indicates group size for weight quantization. EfficientQAT only execute Block-AP and without E2E-QP for the fair comparisons in block-wise reconstruction scenario. Refer Table III for the metric setting.

Model	Method	Precision	PPL	Acc.
3-8B	Baseline	FP16	6.14	72.71
	EfficientQAT	W3A16g128	7.34	70.80
	PrefixQuant	W3A16g128	7.17	72.10
	EfficientQAT	W2A16g128	13.55	58.95
	PrefixQuant	W2A16g128	11.97	64.00
3-70B	Baseline	FP16	2.85	80.03
	EfficientQAT	W3A16g128	4.89	75.65
	PrefixQuant	W3A16g128	4.79	77.75
	EfficientQAT	W2A16g128	16.79	67.88
	PrefixQuant	W2A16g128	11.01	72.61

Performance of Weight-only quantization. In addition to weight-activation quantization, we also explore whether eliminating outlier tokens can enhancing weight-only quantization. To confirm this, we compare our proposed PrefixQuant with EfficientQAT [16] in a block-wise fine-tuning setup. Following

TABLE X: **Ablation study on quantization techniques used in PrefixQuant.** The model used here is Llama-3-8B, and WikiText2 perplexity with 2048 context length is reported. Both PrefixQuant-O1 and PrefixQuant-O2 are start from the “Base”.

	Method	Activation Quant.	W4A8KV4	W4A4KV4
	QuaRot	dynamic	6.75	8.33
Base	RTN	dynamic	12.66	1282.34
	+ rotation	dynamic	10.88	24.98
	+ Grid search initialization	dynamic	9.08	11.70
	+ prefixed outliers	dynamic	6.81	7.53
PrefixQuant-O1	+ block-wise fine-tuning (weight-only)	dynamic	6.70	7.35
	+ block-wise fine-tuning	dynamic	6.59	7.23
	+ static quantization	static	29.07	141.02
PrefixQuant-O2	+ prefixed outliers	static	6.90	7.93
	+ block-wise fine-tuning (weight-only)	static	6.73	7.62
	+ block-wise fine-tuning	static	6.62	7.41

TABLE XI: **Ablation study on prefixed tokens without random Hadamard rotation.**

	Method	Activation Quant.	W4A8KV4	W4A4KV4
	QuaRot	dynamic	6.75	8.33
Base	RTN	dynamic	12.66	1282.34
	+ Grid search initialization	dynamic	8.76	302.06
	+ prefixed outliers	dynamic	7.85	87.83
	+ static quantization	static	53.56	4936.97
PrefixQuant-O2	+ prefixed outliers	static	19.54	424.10

EfficientQAT, we train with 4096 RedPajama [50] samples, each with a 2048 context length, for 2 epochs. The learning rates for quantization parameters and full-precision weights are $5e-5$ and $5e-6$, respectively, except for W2A16g128 Llama-3-8B, where they are $1e-4$ and $2e-5$. As shown in Table IX, **PrefixQuant** outperforms EfficientQAT by +5.05 and +4.73 points in average accuracy on W2A16g128 Llama-3-8B and Llama-3-70B, respectively. This experiment shows that prefixed outlier tokens not only lower activation quantization error but also boost other training-based methods by isolating outlier noise during training.

C. Performance Analysis and Ablation Studies

TABLE XII: **Ablation Study on Prefixed Token Content.** Reports WikiText2 perplexity with a 2048 context length using W4A4KV4 PrefixQuant-O2 without fine-tuning. “Ours” denotes prefixed tokens from the proposed method, “random-k” averages 10 runs with randomly chosen tokens, “repeated top-1” repeats the most frequent outlier token k times, and “Time” measures the time to identify prefixed tokens.

Model	Method	PPL	Time
Llama-2-7B	random-k	66.51	0 s
	repeated top-1	12.07	12 s
	grid search	6.22	3.2 h
	top-k (ours)	6.22	11 s
	random-k	80.05	0 s
Mistral-7B-v0.3	repeated top-1	6.23	12 s
	grid search	5.90	6.8 h
	top-k (ours)	5.89	11 s

Content of refixed token. PrefixQuant identifies k outlier tokens and assigns the top- k frequent ones as prefixes in the KV cache. Table XII compares methods with equal token counts: “random-k” selects k tokens randomly, “repeated top-1” repeats the most frequent outlier k times, and “grid search”

TABLE XIII: Ablation study about the number of prefixed tokens. WikiText2 perplexity with 2048 context length and W4A4KV4 (PrefixQuant-O2) quantization is reported. Number n indicates the first n tokens in Table II are set as the prefixed tokens.

Model	Method	0	1	2	3	4
Llama-2-7B	PrefixQuant	17.63	10.71	6.01	6.01	
Mistral-7B-v0.3	PrefixQuant	15.97	7.08	5.83	5.95	5.79

optimizes tokens across the vocabulary similar to Cushion-Cache [26]. Results show that top- k frequent tokens yield the lowest perplexity, matching grid search (6.22 for Llama-2-7B, 5.89 for Mistral-7B-v0.3) but in only **11** seconds versus 3.2–6.8 hours. Random-k and repeated top-1 perform worse.

Number of Prefixed Tokens. In Sec. IV-B, we determine the number of prefixed tokens by calculating the average number of outlier tokens and adding an additional [BOS] token. Table II illustrates the specific number and content of these tokens. We use Llama-2-7B (3 outlier tokens) and Mistral-7B-v0.3 (4 outlier tokens) to study the impact of the number of prefixed tokens. Table XIII shows that the adaptively calculated number of prefixed tokens achieves the best performance. Notably, for models like Llama-2-7B, using 2 prefixed tokens without the additional [BOS] token also yields excellent performance. For consistency and simplicity, we include the [BOS] token in the prefixed tokens in our experiments.

Effects of each component. We analyze the effects of various quantization techniques implemented in PrefixQuant. These techniques are applied incrementally, and the WikiText2 perplexity results are presented in Table X. The analysis begins with round-to-nearest (RTN) quantization on Llama-3-8B, incorporating Hadamard rotation and grid search initialization.

Using W4A4KV4 as an example, we observe that introducing prefixed outliers significantly improves performance. Specifically, perplexity decreases from 11.70 to 7.53 for PrefixQuant-O1 and from 141.02 to 7.93 for PrefixQuant-O2. These improvements result not only from mitigating information loss caused by outlier tokens but also from enabling more accurate quantization parameter selection during grid searches initialization by isolating extremely large outliers (e.g., values exceeding $1e3$) in activations. Additionally, block-wise fine-tuning further enhances performance, reducing perplexity by 0.30 for PrefixQuant-O1 and by 0.52 for PrefixQuant-O2 in W4A4KV4 quantization. Although our block-wise fine-tuning builds on EfficientQAT [16], PrefixQuant also introduces trainable activation-quantization parameters. Table X quantifies their effect: the row “+ block-wise fine-tuning (weight-only)” omits these activation-quantization parameters. Removing them consistently increases perplexity by 0.11–0.21. Additional ablation results, including analyses of the training dataset, training epochs, dynamic quantization, the number of prefixed tokens, and the content of prefixed tokens, are provided in the Appendix.

TABLE XIV: Inference speedup of W4A4 Llama-2-7B model over the FP16 model on RTX 3090 GPU. For prefilling, we report the latency to deal with 2048 input tokens. For decoding, we report the token generation speed of generate 256 new tokens with 2048 prefilling length.

Method	prefilling (ms)	decoding (token/s)
FP16	489	43
PrefixQuant-O1 (W4A4)	183 (2.67x)	91 (2.11x)
PrefixQuant-O2 (W4A4)	178 (2.74x)	93 (2.16x)

Effects of prefixed tokens without random Hadamard Rotation. Table XI presents the ablation results for the prefixed tokens after removing the random Hadamard Rotation. The findings show that the proposed prefixed tokens still significantly improve perplexity, as they address token-wise outliers. However, the performance degenerates considerably when random Hadamard rotation is removed. These results indicate that both token-wise and channel-wise outliers must be considered to achieve better quantized models. Furthermore, the proposed PrefixQuant can also serve as a plug-and-play enhancement for other methods designed to address channel-wise outliers.

D. Inference Speed

In this section, we evaluate the end-to-end inference speed of PrefixQuant under the W4A4 quantization scenario. KV quantization is not considered because it reduces memory usage at the cost of increased computation overhead and only provides speedup with large batch sizes or extremely long input sequence [28]. To maximize end-to-end inference speed, we compile the entire quantized model into a CUDA graph with `torch.compile`, which minimizes CUDA kernel-launch overhead. For quantized linear layers, we use a dual-kernel strategy based on the input case. For batched inputs

(batch size > 1), we use a high-throughput 4-bit GEMM kernel from the CUTLASS library, tuning tile sizes per layer to improve hardware utilization. For latency-critical inputs with batch size = 1, we present a fused GEMV–dequantization kernel that reduces off-chip memory traffic by performing dequantization inside the matrix–vector multiply and exploits vectorized memory access for higher efficiency. As shown in Table XIV, PrefixQuant achieves an approximate $2.7\times$ speedup in prefilling and a $2.1\times$ speedup in decoding compared to the FP16 model. Additionally, offline static quantization makes PrefixQuant-O2 3% faster in prefilling and 2% faster in decoding than PrefixQuant-O1.

E. Quantization Time

Table XV shows the quantization time for PrefixQuant. PrefixQuant identifies prefixed tokens quickly, taking only 0.2 minutes for Llama-3-8B and 1 minute for Llama-3-70B. In contrast, the recent CushionCache [26] requires 12 hours for the same task on Llama-3-8B. Additionally, the grid-search initialization is efficient, taking 0.7 minutes for Llama-3-8B and 12 minutes for Llama-3-70B. Experiments in Tables III and IV demonstrate that PrefixQuant, even without fine-tuning, outperforms previous methods [7], [17]. Fine-tuning requires more time, taking 2.2 hours for Llama-3-8B and 17 hours for Llama-3-70B, but it can successfully enhance the potential of low-bit quantization.

TABLE XV: The quantization time of PrefixQuant on single NVIDIA-A100-80GB GPU. Fine-tuning indicates the time of 20 fine-tuning epochs of W4A4KV4.

Model	Prefixed Outliers	Grid-search Init.	Fine-tuning
Llama-3-8B	0.2 m	0.7 m	2.2 h
Llama-3-70B	1 m	12 m	17 h

VI. CONCLUSION

In this paper, we propose PrefixQuant, which provides a comprehensive exploration of outlier tokens and introduces an efficient and effective method to handle them by prefixing these tokens in the KV cache. Additionally, we design new trainable parameters for activation quantization to minimize quantization error. The proposed PrefixQuant method achieves excellent performance across various models, quantization precisions, and granularities. The simplicity and broad applicability of PrefixQuant make it a promising direction for future research on LLM compression and optimization.

VII. LIMITATION

While PrefixQuant demonstrates significant improvements in LLM quantization, we acknowledge several limitations that suggest avenues for future research. Although identifying prefixed tokens is highly efficient, PrefixQuant relies on a block-wise fine-tuning stage to achieve optimal performance. This step adds a layer of complexity compared to methods that are entirely training-free. Additionally, the performance of PrefixQuant on other architectures, such as Mixture-of-Experts (MoE) models, remains an open area for investigation.

VIII. ACKNOWLEDGEMENT

This paper is partially supported by the National Key R&D Program of China No.2022ZD0161000 and the General Research Fund of Hong Kong No.17200622.

REFERENCES

- [1] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar *et al.*, “Llama: Open and efficient foundation language models,” *arXiv preprint arXiv:2302.13971*, 2023.
- [2] S. Bubeck, V. Chandrasekaran, R. Eldan, J. Gehrke, E. Horvitz, E. Kamar, P. Lee, Y. T. Lee, Y. Li, S. Lundberg *et al.*, “Sparks of artificial general intelligence: Early experiments with gpt-4,” *arXiv preprint arXiv:2303.12712*, 2023.
- [3] E. Frantar, S. Ashkboos, T. Hoeffer, and D. Alistarh, “GPTQ: Accurate post-training compression for generative pretrained transformers,” in *The Eleventh International Conference on Learning Representations*, 2023.
- [4] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, “Awq: Activation-aware weight quantization for on-device llm compression and acceleration,” *Proceedings of machine learning and systems*, vol. 6, pp. 87–100, 2024.
- [5] W. Shao, M. Chen, Z. Zhang, P. Xu, L. Zhao, Z. Li, K. Zhang, P. Gao, Y. Qiao, and P. Luo, “OmniQuant: Omnidirectionally calibrated quantization for large language models,” in *The Twelfth International Conference on Learning Representations*, 2024.
- [6] Z. Yuan, Y. Shang, Y. Zhou, Z. Dong, C. Xue, B. Wu, Z. Li, Q. Gu, Y. J. Lee, Y. Yan *et al.*, “Llm inference unveiled: Survey and roofline model insights,” *arXiv preprint arXiv:2402.16363*, 2024.
- [7] S. Ashkboos, A. Mohtashami, M. L. Croci, B. Li, P. Cameron, M. Jaggi, D. Alistarh, T. Hoeffer, and J. Hensman, “Quarot: Outlier-free 4-bit inference in rotated llms,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 100 213–100 240, 2024.
- [8] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, “Gpt3. int8 (): 8-bit matrix multiplication for transformers at scale,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 30 318–30 332, 2022.
- [9] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, “SmoothQuant: Accurate and efficient post-training quantization for large language models,” in *International Conference on Machine Learning*. PMLR, 2023, pp. 38 087–38 099.
- [10] X. Wei, Y. Zhang, Y. Li, X. Zhang, R. Gong, J. Guo, and X. Liu, “Outlier suppression+: Accurate quantization of large language models by equivalent and effective shifting and scaling,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 1648–1665.
- [11] Y. Zhao, C.-Y. Lin, K. Zhu, Z. Ye, L. Chen, S. Zheng, L. Ceze, A. Krishnamurthy, T. Chen, and B. Kasikci, “Atom: Low-bit quantization for efficient and accurate llm serving,” *Proceedings of Machine Learning and Systems*, vol. 6, pp. 196–209, 2024.
- [12] Z. Liu, C. Zhao, I. Fedorov, B. Soran, D. Choudhary, R. Krishnamoorthi, V. Chandra, Y. Tian, and T. Blankevoort, “SpinQuant: Llm quantization with learned rotations,” in *The Thirteenth International Conference on Learning Representations*, 2025.
- [13] J. Liu, R. Gong, X. Wei, Z. Dong, J. Cai, and B. Zhuang, “QlLM: Accurate and efficient low-bitwidth quantization for large language models,” in *International Conference on Learning Representations 2024*. International Conference on Learning Representations (ICLR), 2024.
- [14] M. Sun, X. Chen, J. Z. Kolter, and Z. Liu, “Massive activations in large language models,” in *First Conference on Language Modeling*, 2024.
- [15] Y. Bondarenko, M. Nagel, and T. Blankevoort, “Quantizable transformers: Removing outliers by helping attention heads do nothing,” *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [16] M. Chen, W. Shao, P. Xu, J. Wang, P. Gao, K. Zhang, Y. Qiao, and P. Luo, “EfficientQat: Efficient quantization-aware training for large language models,” *arXiv preprint arXiv:2407.11062*, 2024.
- [17] Y. Lin, H. Tang, S. Yang, Z. Zhang, G. Xiao, C. Gan, and S. Han, “Qserve: W4a8kv4 quantization and system co-design for efficient llm serving,” in *Eighth Conference on Machine Learning and Systems*, 2024.
- [18] S. Ashkboos, I. Markov, E. Frantar, T. Zhong, X. Wang, J. Ren, T. Hoeffer, and D. Alistarh, “Quik: Towards end-to-end 4-bit inference on generative large language models,” in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024, pp. 3355–3371.
- [19] X. Wei, Y. Zhang, X. Zhang, R. Gong, S. Zhang, Q. Zhang, F. Yu, and X. Liu, “Outlier suppression: Pushing the limit of low-bit transformer language models,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 17 402–17 414, 2022.
- [20] X. Wei, Y. Zhang, Y. Li, X. Zhang, R. Gong, J. Guo, and X. Liu, “Outlier suppression+: Accurate quantization of large language models by equivalent and effective shifting and scaling,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 1648–1665.
- [21] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, “Efficient streaming language models with attention sinks,” in *The Twelfth International Conference on Learning Representations*, 2024.
- [22] X. Gu, T. Pang, C. Du, Q. Liu, F. Zhang, C. Du, Y. Wang, and M. Lin, “When attention sink emerges in language models: An empirical view,” in *The Thirteenth International Conference on Learning Representations*, 2025.
- [23] C. Han, Q. Wang, W. Xiong, Y. Chen, H. Ji, and S. Wang, “Lm-infinite: Simple on-the-fly length generalization for large language models,” *arXiv preprint arXiv:2308.16137*, 2023.
- [24] R. Liu, H. Bai, H. Lin, Y. Li, H. Gao, Z. Xu, L. Hou, J. Yao, and C. Yuan, “Intactkv: Improving large language model quantization by keeping pivot tokens intact,” in *Findings of the Association for Computational Linguistics ACL 2024*, 2024, pp. 7716–7741.
- [25] J. Yang, H. Kim, and Y. Kim, “Mitigating quantization errors due to activation spikes in glu-based llms,” *arXiv preprint arXiv:2405.14428*, 2024.
- [26] S. Son, W. Park, W. Han, K. Kim, and J. Lee, “Prefixing attention sinks can mitigate activation outliers for large language model quantization,” in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024, pp. 2242–2252.
- [27] J. Jiang, P. Yang, R. Zhang, and F. Liu, “Towards efficient large language model serving: A survey on system-aware kv cache optimization,” *Authorea Preprints*, 2025.
- [28] Z. Liu, J. Yuan, H. Jin, S. Zhong, Z. Xu, V. Braverman, B. Chen, and X. Hu, “Kivi: a tuning-free asymmetric 2bit quantization for kv cache,” in *Proceedings of the 41st International Conference on Machine Learning*, 2024, pp. 32 332–32 344.
- [29] Y. He, L. Zhang, W. Wu, J. Liu, H. Zhou, and B. Zhuang, “Zipcache: Accurate and efficient kv cache quantization with salient token identification,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 68 287–68 307, 2024.
- [30] Y. Zhang, Y. Du, G. Luo, Y. Zhong, Z. Zhang, S. Liu, and R. Ji, “Cam: Cache merging for memory-efficient llms inference,” in *Forty-first international conference on machine learning*, 2024.
- [31] R. Zhang, K. Wang, L. Liu, S. Wang, H. Cheng, C. Zhang, and Y. Shen, “Lorc: Low-rank compression for llms kv cache with a progressive compression strategy,” *arXiv preprint arXiv:2410.03111*, 2024.
- [32] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett *et al.*, “H2o: Heavy-hitter oracle for efficient generative inference of large language models,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 34 661–34 710, 2023.
- [33] Y. Li, Y. Huang, B. Yang, B. Venkitesh, A. Locatelli, H. Ye, T. Cai, P. Lewis, and D. Chen, “Snapkv: Llm knows what you are looking for before generation,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 22 947–22 970, 2024.
- [34] I. Gim, G. Chen, S.-s. Lee, N. Sarda, A. Khandelwal, and L. Zhong, “Prompt cache: Modular attention reuse for low-latency inference,” *Proceedings of Machine Learning and Systems*, vol. 6, pp. 325–338, 2024.
- [35] L. Ye, Z. Tao, Y. Huang, and Y. Li, “Chunkattention: Efficient self-attention with prefix-aware kv cache and two-phase partition,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 11 608–11 620.
- [36] M. Nagel, M. Fournarakis, R. A. Amjad, Y. Bondarenko, M. Van Baalen, and T. Blankevoort, “A white paper on neural network quantization,” *arXiv preprint arXiv:2106.08295*, 2021.
- [37] Y. Li, R. Gong, X. Tan, Y. Yang, P. Hu, Q. Zhang, F. Yu, W. Wang, and S. Gu, “Brecq: Pushing the limit of post-training quantization by block reconstruction,” *arXiv preprint arXiv:2102.05426*, 2021.
- [38] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, “Roformer: Enhanced transformer with rotary position embedding,” *Neurocomputing*, vol. 568, p. 127063, 2024.
- [39] H. Lin, H. Xu, Y. Wu, J. Cui, Y. Zhang, L. Mou, L. Song, Z. Sun, and Y. Wei, “Duquant: Distributing outliers via dual transformation makes stronger quantized llms,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 87 766–87 800, 2024.

- [40] X. Hu, Y. Cheng, D. Yang, Z. Chen, Z. Xu, Z. Yuan, S. Zhou *et al.*, “Ostquant: Refining large language model quantization with orthogonal and scaling transformations for better distribution fitting,” in *The Thirteenth International Conference on Learning Representations*, 2025.
- [41] Y. Sun, R. Liu, H. Bai, H. Bao, K. Zhao, Y. Li, X. Yu, L. Hou, C. Yuan, X. Jiang *et al.*, “Flatquant: Flatness matters for llm quantization,” in *Forty-second International Conference on Machine Learning*, 2025.
- [42] S. Merity, C. Xiong, J. Bradbury, and R. Socher, “Pointer sentinel mixture models,” *arXiv preprint arXiv:1609.07843*, 2016.
- [43] Y. Bisk, R. Zellers, J. Gao, Y. Choi *et al.*, “Piqa: Reasoning about physical commonsense in natural language,” in *Proceedings of the AAAI conference on artificial intelligence*, 2020, pp. 7432–7439.
- [44] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord, “Think you have solved question answering? try arc, the ai2 reasoning challenge,” *arXiv preprint arXiv:1803.05457*, 2018.
- [45] R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi, “Hellaswag: Can a machine really finish your sentence?” in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019, pp. 4791–4800.
- [46] K. Sakaguchi, R. L. Bras, C. Bhagavatula, and Y. Choi, “Winogrande: An adversarial winograd schema challenge at scale,” *Communications of the ACM*, vol. 64, no. 9, pp. 99–106, 2021.
- [47] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt, “Measuring massive multitask language understanding,” *arXiv preprint arXiv:2009.03300*, 2020.
- [48] L. Gao, J. Tow, B. Abbasi, S. Biderman, S. Black, A. DiPofi, C. Foster, L. Golding, J. Hsu, A. Le Noac’h, H. Li, K. McDonell, N. Muennighoff, C. Ociepa, J. Phang, L. Reynolds, H. Schoelkopf, A. Skowron, L. Sutawika, E. Tang, A. Thite, B. Wang, K. Wang, and A. Zou, “A framework for few-shot language model evaluation,” 07 2024. [Online]. Available: <https://zenodo.org/records/12608602>
- [49] L. Gao, S. Biderman, S. Black, L. Golding, T. Hoppe, C. Foster, J. Phang, H. He, A. Thite, N. Nabeshima *et al.*, “The pile: An 800gb dataset of diverse text for language modeling,” *arXiv preprint arXiv:2101.00027*, 2020.
- [50] T. Computer, “Redpajama: an open dataset for training large language models,” 2023. [Online]. Available: <https://github.com/togethercomputer/RedPajama-Data>
- [51] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, “Exploring the limits of transfer learning with a unified text-to-text transformer,” *The Journal of Machine Learning Research*, vol. 21, no. 1, pp. 5485–5551, 2020.

IX. BIOGRAPHY SECTION



Mengzhao Chen is currently a Ph.D. student at the University of Hong Kong (HKU) and supervised by Prof. Ping Luo. Earlier, he received his Bachelor’s and Master’s degree from Xiamen University and supervised by Prof. Rongrong Ji. His current research interest is efficient training and inference of large models. He has published peer-reviewed papers in top-tier conference and journals such as IJCV, ICLR, ICCV, ACL, ECCV, and AAAI.



Yi Liu is an AI frameworks and algorithm engineer at Intel. He received his Master’s degree from the National University of Defense Technology in 2019 and his Bachelor’s degree from East China University of Science and Technology in 2017. His current research interests include efficient inference algorithms and framework development for large language models.



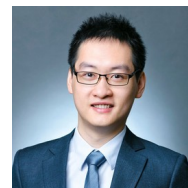
Jiaohao Wang is currently a Ph.D. student at the University of Hong Kong (HKU) and a former full-time researcher at the Shanghai AI Laboratory, where he worked on deep learning algorithms. He received his Master’s degree from Tsinghua University in 2022 and his Bachelor’s degree from Hefei University of Technology in 2019. His current research interests include visual generation and efficient AI.



Yi Bin is a Research Professor with the School of Computer Science and Technology of Tongji University. Before this, he was a Research Fellow at National University of Singapore. He received the Ph.D degree at UESTC at the end of 2020. His research interests focus on multimodal analysis and reasoning. He has published several papers on top-tier conferences and journals, including CVPR, ICLR, AAAI, ACL, ACM MM, IEEE TIP, etc.



Wenqi Shao is a Young Scientist at the Shanghai Artificial Intelligence Laboratory. He completed his Ph.D. in 2022 at the Multimedia Lab of the Chinese University of Hong Kong (CUHK), where he was supervised by Prof. Xiaogang Wang, Prof. Ping Luo, and Prof. Hongsheng Li. Before his doctoral studies, he obtained a bachelor’s degree from the School of Mathematics at the University of Electronic Science and Technology of China (UESTC) in 2017. His research interests revolve around multimodal foundation models, large language model compression, efficient transfer learning, and their applications in multimedia. Wenqi Shao is actively involved in the academic community and serves as a reviewer for several prestigious conferences, including CVPR, ICCV, ICML, NeurIPS, and ICLR.



Ping Luo is an Associate Professor in the Department of Computer Science at the University of Hong Kong (HKU). He received his PhD degree in 2014 in Information Engineering, from the Chinese University of Hong Kong (CUHK), supervised by Prof. Xiaou Tang and Prof. Xiaogang Wang. He was a Postdoctoral Fellow at CUHK from 2014 to 2016. He joined SenseTime Research as a Principal Research Scientist from 2017 to 2018. His research interests are machine learning and computer vision.

He has published 100+ peer-reviewed articles in top-tier conferences and journals such as TPAMI, IJCV, ICML, ICLR, CVPR, and NIPS.